

一、个人项目背景

我选择的是大学期间独立完成的「校园闲置物品交易小程序」项目，核心功能包含用户发布 / 浏览闲置、在线沟通、订单管理、校园自提点预约，是一个面向本校学生的轻量化二手交易平台，开发周期约 8 周。

二、项目软件过程定义

结合项目规模与个人开发场景，我定义了一套适配单人开发的轻量软件过程，分为以下 6 个阶段：

表格

阶段	核心活动	交付物	完成标准
1. 需求分析与规划	1. 调研目标用户（本校学生）核心痛点	《需求说明书》《项目排期表》	明确 3 个核心模块，排期拆解到周，无模糊需求
	2. 梳理核心功能与非功能需求		
	3. 制定开发周期与里程碑		
2. 原型设计与架构设计	1. 绘制产品原型图（首页、发布页、聊天页）	原型图、架构图、数据库设计文档	原型覆盖所有核心流程，架构满足并发需求
	2. 设计前后端技术架构（小程序 + SpringBoot+MySQL）		
	3. 定义数据库表结构		
3. 开发实现	1. 按模块迭代开发（用户模块→商品模块→订单模块→聊天模块）	各模块可运行代码、版本提交记录	每个模块自测通过，无严重功能 bug
	2. 每日本地自测基础功能		
	3. 用 Git 做版本管理，按功能提交代码		
4. 单元测试与集成测试	1. 对核心接口编写单元测试	单元测试报告、测试反馈记录	核心流程无阻塞性 bug，问题记录完整
	2. 模块间联调，验证流程闭环（发布→沟通→下单）		
	3. 邀请同学进行灰度测试，收集反馈		

阶段	核心活动	交付物	完成标准
5. 部署与上线	1. 后端部署到云服务器，配置域名与 HTTPS	上线版本、使用手册、运维文档	小程序审核通过，可正常访问使用
	2. 小程序提交微信公众平台审核		
	3. 编写用户使用说明与运维手册		
6. 运维与迭代	1. 监控服务器运行状态与用户反馈	运维日志、迭代优化方案	线上服务稳定运行，bug 响应不超过 24 小时
	2. 修复线上 bug，优化体验（如图片加载速度）		
	3. 规划下一版本功能（如信用评价）		

三、软件过程的来源与剪裁思路

1. 过程定义的核心来源

我的软件过程主要参考了两个成熟的软件工程模型：

- 瀑布模型：保留了线性阶段划分（需求→设计→开发→测试→部署），保证每个阶段有明确的交付物和完成标准，避免单人开发中出现“想到哪写到哪”的混乱。
- 敏捷开发思想：吸收了迭代开发的理念，没有采用瀑布模型的一次性全流程开发，而是按模块分阶段迭代，每完成一个模块就进行自测和调整，降低后期返工的风险。

2. 针对个人项目的剪裁思路

原始的软件工程模型（如瀑布模型、RUP）大多是为团队大型项目设计的，流程复杂、文档要求高，直接套用会导致单人开发效率极低，因此我做了以下关键剪裁：

表格

原始模型的冗余环节	我的剪裁方式	剪裁原因
团队角色分工（产品经理、测试工程师、运维工程师）	单人兼任所有角色，交叉职责	项目仅 1 人开发，无需复杂的角色流程，减少沟通成本
大规模需求评审、架构评审会议	简化为个人需求梳理 + 原型验证	无团队协作对象，通过用户调研和原型自测即可确认需求，无需

原始模型的冗余环节	我的剪裁方式	剪裁原因
		多人评审
完整的项目管理流程 (周报、里程碑评审)	仅保留周级里程碑和每日代码提交记录	单人项目无需正式汇报，通过 Git 提交记录即可追踪进度，避免形式化流程
全面的系统测试与验收测试	简化为单元测试 + 核心流程集成测试 + 小范围用户灰度测试	项目用户量小，核心流程稳定即可满足需求，无需大规模测试
复杂的配置管理与变更控制流程	仅使用 Git 进行版本控制，关键节点打标签	单人开发无多分支冲突问题，无需复杂的变更审批流程

3. 剪裁的核心原则

1. 适配项目规模：过程的复杂度与项目规模匹配，不做“大项目的小流程”，避免流程大于项目本身。
2. 保留核心价值：所有剪裁都不破坏软件工程的核心逻辑 —— 明确需求、设计先行、测试验证、版本可控。
3. 提升开发效率：去掉所有形式化、无实际产出的环节，把精力集中在编码和核心交付上。